

技術文件（繁體中文）

專案：Download Bank of Taiwan Exchange Rate

對象：開發 / 維運 / NetSuite 技術人員（需能自行修改邏輯）

英文版：TECHNICAL.en.md (./TECHNICAL.en.md)

使用文件：USER_GUIDE.zh-TW.md (./USER_GUIDE.zh-TW.md)

1. 目標與設計原則

原則	說明
不讓 NetSuite 直連台銀	台銀有 Akamai 類 bot 防護，N/https 常失敗
CDN 提供穩定 HTTPS	使用 GitHub public repo + jsDelivr
JSON 優先	NetSuite 端 JSON.parse 即可，不需解析 CSV
可改、可擴充	下載、轉換、排程、NetSuite 寫入皆模組化

資料流：

```
rate.bot.com.tw CSV
  |
  ▼
scripts/download_bot_rates.py
- curl_cffi 下載 ( 模擬瀏覽器 TLS )
- CSV → JSON 轉換
- 寫入 data/
  |
  ▼
GitHub Actions (.github/workflows/update-rates.yml)
- 平日排程 / 手動觸發
- 有變更才 commit + push
- purge jsDelivr
  |
  ▼
cdn.jsdelivr.net/.../bot-xrt-latest.json
  |
  ▼
netsuite/DownloadBotRates_SS2.js
- N/https.get
- JSON.parse
- 存 File Cabinet ( 可擴充寫入 Currency Rate )
```

2. 目錄結構

```
DownloadBankOfTaiwanExchangeRate/
├── .github/workflows/update-rates.yml # 排程與發布
├── data/
│   ├── bot-xrt-latest.csv          # 最新原始 CSV
│   ├── bot-xrt-latest.json         # 最新 JSON ( CDN 主檔 )
│   ├── bot-xrt-latest.meta.txt     # checksum / 抓取時間
│   └── history/                   # 歷史歸檔 ( 只留最近 90 天 )
```

```
| └─ bot-xrt-YYYY-MM-DD.*  
| └─ docs/                # 本文件區  
| └─ netsuite/DownloadBotRates_SS2.js  # NetSuite Scheduled Script  
| └─ scripts/download_bot_rates.py    # 下載 + 轉換核心  
| └─ requirements.txt  
| └─ README.md
```

3. 本機開發與手動執行

3.1 環境

- Python 3.10+ (CI 使用 3.12)
- 相依 : curl_cffi (見 requirements.txt)

```
python3 -m pip install -r requirements.txt  
python3 scripts/download_bot_rates.py
```

3.2 CLI 參數

參數	說明
-o / --output	指定 bot-xrt-latest.csv 路徑 ; JSON 會寫在同目錄
--no-archive	不寫 data/history/bot-xrt-YYYY-MM-DD.* 歸檔
--retention-days	歷史保留天數 (預設 90)

3.3 成功輸出範例

```
csv=.../data/bot-xrt-latest.csv  
json=.../data/bot-xrt-latest.json  
csv_bytes=3715  
json_bytes=22669  
rate_count=19  
changed=true  
archive_json=.../data/history/bot-xrt-2026-08-11.json  
retention_days=90  
history_migrated=0  
history_pruned=0
```

歷史策略 :

- 歸檔目錄 : data/history/
- 檔名 : bot-xrt-YYYY-MM-DD.csv / .json
- 預設只保留最近 90 天 ; 超過會自動刪除
- 若 data/ 根目錄仍有舊的日期檔 , 執行時會自動搬到 history/

changed=false 表示內容與上次相同 , 檔案未覆寫 (meta 仍會更新) 。

4. 核心腳本 : scripts/download_bot_rates.py

4.1 函式一覽 (修改入口)

函式	職責	何時改它
download_csv()	從台銀抓 CSV	換
_to_rate()	字串→數字 ; 0→null	URL、headers、timeout、impersonate 想保留 0、改精度、過濾規則
_forward_block()	組遠期 10 ~ 180 天	增減 tenor
csv_to_payload()	CSV→JSON 結構	改 JSON schema 的主要入口
write_outputs()	寫 CSV/JSON/meta/history	改檔名、歸檔策略、變更偵測
migrate_legacy_archives()	把舊日期檔搬到 history/	相容舊結構
prune_history()	刪除超過保留天數的歷史檔	改保留天數邏輯
main()	CLI	加參數、加後處理

4.2 關鍵常數

```
BOT_CSV_URL = "https://rate.bot.com.tw/xrt/flcsv/0/day"
FORWARD_DAYS = (10, 30, 60, 90, 120, 150, 180)
```

```
BUY_CASH = 2
BUY_SPOT = 3
BUY_FWD_START = 4
SELL_CASH = 12
SELL_SPOT = 13
SELL_FWD_START = 14
```

4.3 台銀 CSV 欄位對照 (0-based)

每一資料列大致為：

Index	內容
0	幣別 (USD)
1	標籤「本行買入」
2	現金買入
3	即期買入
4-10	遠期買入 10/30/60/90/120/150/180
11	標籤「本行賣出」
12	現金賣出
13	即期賣出
14-20	遠期賣出 10/30/60/90/120/150/180

若台銀調整欄位順序，只改常數與 csv_to_payload() 即可，不必動 NetSuite。

4.4 為什麼用 curl_cffi？

一般 requests / urllib 的 TLS 指紋容易被擋，會拿到 Challenge HTML。

curl_cffi 以 impersonate="chrome" 模擬瀏覽器握手。

若再次被擋，可依序嘗試：

1. 換 impersonate (chrome / chrome131 / safari)

2. 調整 Referer / Accept-Language
3. 最後才考慮 Playwright (較重 , CI 成本高)

偵測失敗條件 (請保留) :

- Content-Type 含 text/html
- body 以 <!DOCTYPE 開頭
- 前 200 bytes 沒有逗號 (不像 CSV)

4.5 JSON Schema (契約)

```
{
  "source": "Bank of Taiwan",
  "sourceUrl": "https://rate.bot.com.tw/xrt/flcsv/0/day",
  "base": "TWD",
  "fetchedAtUtc": "ISO-8601",
  "rateCount": 19,
  "rates": [ { "...": "CurrencyEntry" } ],
  "byCurrency": {
    "USD": { "...": "CurrencyEntry" }
  }
}
```

CurrencyEntry :

```
{
  "currency": "USD",
  "cash": { "buy": 31.87, "sell": 32.54 },
  "spot": { "buy": 32.195, "sell": 32.345 },
  "forward": {
    "buy": { "10": 32.214, "30": 32.164, "60": null, "90": null, "120": null, "150": null, "180": null },
    "sell": { "10": 32.318, "30": 32.272, "60": null, "90": null, "120": null, "150": null, "180": null }
  }
}
```

契約注意 :

- NetSuite 依賴 byCurrency 與 rates 同時存在
- 無報價用 null , 不要改回 0 , 除非同步改 NetSuite 判斷
- 新增欄位 (例如 mid、幣別中文名) 建議加在 CurrencyEntry , 保持向後相容

4.6 變更偵測

write_outputs() 比對 :

- bot-xrt-latest.csv 位元組是否改變
- bot-xrt-latest.json 位元組是否改變

任一改變 → changed=true。

注意 : fetchedAtUtc 每次都會變 , 因此 JSON 幾乎每次都會不同 , 即使匯率相同。

若希望「匯率不變就不發布」, 可改成 :

1. 產生 payload 時先拿掉 / 固定 fetchedAtUtc 再比對
2. 或比對 byCurrency 內容 (建議用 canonical JSON)

範例修改方向 (概念) :

```
comparable = {k: v for k, v in payload.items() if k != "fetchAtUtc"}
new_bytes = json.dumps(comparable, ensure_ascii=False, sort_keys=True).encode()
# 與上次 comparable 比對 ; 真正寫入時仍可帶新的 fetchAtUtc
```

5. GitHub Actions : .github/workflows/update-rates.yml

5.1 觸發

```
on:
  workflow_dispatch:      # 主要 : 外部 cron / 手動
  repository_dispatch:
    types: [update-bot-rates]
  schedule:               # 僅備援 ( UTC )
    - cron: "30 2,7,10 * * 1-5" # 台灣 10:30 / 15:30 / 18:30
```

請以外部 cron 為主要觸發 (GitHub 內建 schedule 容易漏跑) 。
完整設定步驟見 SCHEDULING.zh-TW.md (./SCHEDULING.zh-TW.md)。

本機觸發 :

```
./scripts/trigger_update.sh
```

5.2 如何改排程

需求	改法
改外部執行時間	到 cron-job.org (或你的 crontab) 調整
改 GitHub 備援	編輯 schedule.cron (UTC)
暫時只手動	拿掉外部 cron ; 必要時保留/移除 schedule

5.3 Job 步驟與可改點

1. checkout
2. setup-python + pip install
3. 執行 download_bot_rates.py
4. git diff 判斷 data/bot-xrt-latest.csv/json 是否變
5. 有變才 commit / push
6. purge jsDelivr

權限 :

```
permissions:
  contents: write
```

若改用 fine-grained token 或保護分支 , 需同步調整 repo 設定。

5.4 手動 purge CDN

```
curl -fsS "https://purge.jsdelivr.net/gh/ChesleyLo/DownloadBankOfTaiwanExchangeRate@main/data/bot-xrt-latest.json"
```

5.5 Repo 必須 Public

jsDelivr 無法讀 private repo。公開指令：

```
gh repo edit ChesleyLo/DownloadBankOfTaiwanExchangeRate \
--visibility public \
--accept-visibility-change-consequences
```

6. NetSuite : netsuite/DownloadBotRates_SS2.js

6.1 模組與執行點

- Type : Scheduled Script 2.1
- Entry : execute(context)
- Modules : N/https、N/file、N/log、N/runtime

6.2 常數 (常改)

常數	意義
CDN_JSON_URL	預設 CDN
FOLDER_ID	File Cabinet 資料夾 internal id
FILE_NAME	最新檔名
DEFAULT_RATE_FIELD	例如 spot.sell

6.3 函式一覽 (修改入口)

函式	職責	何時改
getConfiguredUrl()	讀 script param 或預設 URL	多環境 URL
getRateFieldPath()	決定用 cash/spot 哪一個	會計政策變更
parsePayload()	驗證 JSON 契約	schema 變更時同步
readRate()	用 "spot.sell" 路徑取值	支援 forward.sell.30 等
saveJson()	寫 File Cabinet	改命名、覆蓋策略
maybeApplyRates()	寫入匯率的擴充點	接 Currency Rate / 自訂記錄
execute()	主流程	加通知、錯誤處理、重試

6.4 建議 Script Parameters

在 Script 定義新增：

ID	Type	說明
custscript_bot_cdn_json_url	Text	CDN URL
custscript_bot_rate_field	Text	spot.sell / cash.buy...
custscript_bot_apply_rates	Checkbox	開啟寫入邏輯

6.5 如何改成真正寫入 NetSuite 匯率

目前 maybeApplyRates() 只 log.debug。要寫入時：

1. define 加入 "N/record" (及必要時 N/search、N/format)
2. 依貴帳號 Currency Rate 記錄欄位填值
3. 注意：多幣別帳、匯率日期、反向匯率 (TWD→USD vs USD→TWD)

概念範例 (需依帳號欄位調整，不可直接套用)：

```
define(["N/https", "N/file", "N/log", "N/runtime", "N/record"], (
  https, file, log, runtime, record
) => {
  // ...
  function upsertCurrencyRate(fromCode, toCode, exchangeRate, effectiveDate) {
    const rec = record.create({ type: record.Type.CURRENCY_RATE, isDynamic: true });
    // rec.setValue({ fieldId: "...", value: ... }); // 依帳號實際欄位
    return rec.save();
  }
});
```

readRate() 已支援點路徑，遠期可傳 forward.sell.30。

6.6 只取部分幣別

在 maybeApplyRates() 內加白名單：

```
const ALLOWED = ["USD", "EUR", "JPY", "CNY"];
currencies.filter((c) => ALLOWED.indexOf(c) !== -1).forEach(/* ... */);
```

6.7 錯誤處理建議

已有：

- HTTP 非 2xx → throw
- JSON 無效 → throw
- 缺少 byCurrency/rates → throw

可再加：

- 重試 2 ~ 3 次 (CDN 短暫失敗)
- 失敗時寄 email / 寫 custom record
- 備援改打 GitHub Raw URL

7. 常見修改情境 (Cookbook)

7.1 新增 JSON 欄位 mid = (buy+sell)/2

在 csv_to_payload() 組好 entry 後：

```
def _mid(side: dict) -> float | None:
```

```
b, s = side.get("buy"), side.get("sell")
if b is None or s is None:
    return None
return round((b + s) / 2, 6)

entry["spot"]["mid"] = _mid(entry["spot"])
entry["cash"]["mid"] = _mid(entry["cash"])
```

NetSuite 端可用 spot.mid。

7.2 改用歷史日期 CSV

台銀亦可能提供：

```
https://rate.bot.com.tw/xrt/flcsv/0/YYYY-MM-DD
```

把 BOT_CSV_URL 改成參數化：

```
parser.add_argument("--date", help="YYYY-MM-DD; default=day")
# url = f"https://rate.bot.com.tw/xrt/flcsv/0/{args.date or 'day'}"
```

7.3 發布到別的 CDN / bucket

Actions 最後一步改成上傳 S3/R2/Azure，並更新 NetSuite URL。
JSON 契約盡量不變，NetSuite 幾乎不用改。

7.4 變更檔名

同步修改：

1. write_outputs() 輸出檔名
2. Actions purge 路徑
3. NetSuite CDN_JSON_URL / FILE_NAME
4. 使用文件中的 URL

8. 測試清單

下載 / 轉換

- [] python3 scripts/download_bot_rates.py 成功
- [] JSON 可 json.load
- [] byCurrency.USD.spot.sell 為 number
- [] 無報價幣別為 null
- [] 故意離線 / 被擋時，exit code ≠ 0

Actions

- [] Actions → Run workflow 成功

- [] data/bot-xrt-latest.json 有更新 commit
- [] CDN URL HTTP 200
- [] purge 後內容為新版本

NetSuite

- [] Execute Now 成功
- [] File Cabinet 出現 latest + dated 檔
- [] Execution Log 有 USD spot sell
- [] (若啟用寫入) Currency Rate / 自訂記錄正確

9. 故障排除

症狀	可能原因	處理
下載到 HTML Challenge	WAF	確認 curl_cffi、換 impersonate
CDN 404 file not found	repo private 或路徑錯	改 public、檢查路徑
CDN 舊資料	cache	purge；或暫用 Raw URL
Actions 無法 push	token/權限	contents: write、分支保護
NetSuite SSL/連線失敗	防火牆 allowlist	放行 cdn.jsdelivr.net
JSON parse 失敗	拿到 HTML/錯誤頁	檢查 URL、HTTP code

10. 安全與合規

- 不要把 secrets 寫進 public repo
- 匯率為公開資訊，但仍應標示來源「台灣銀行」
- CDN 為鏡像，可能延遲；關鍵財務用途請訂對帳 / 覆核流程
- NetSuite 寫入匯率屬財務敏感，變更需 Code Review

11. 版本與維護建議

1. 改 JSON 契約時：先加欄位、後刪欄位 (向後相容)
2. 重大變更更新 docs/ 雙語文件與 README
3. 台銀若改 CSV 格式，優先修 Python，再驗證 NetSuite 無需改動
4. 定期 (每季) 抽查 CDN 與官方頁面數值一致性

12. 文件索引

文件	說明
TECHNICAL.zh-TW.md (./TECHNICAL.zh-TW.md)	本技術文件 (中文)
TECHNICAL.en.md (./TECHNICAL.en.md)	Technical guide (English)

USER_GUIDE.zh-TW.md (./USER_GUIDE.zh-TW.md)	使用說明 (中文)
USER_GUIDE.en.md (./USER_GUIDE.en.md)	User guide (English)